Technical Report — Case Study 1

Title: AUTOSAR HLD AI Assistant: Grounded AI for High-Level Design Analysis, Engineering Q&A and Architecture Exploration

University: [TO BE FILLED] Student: [TO BE FILLED] Register No.: [TO BE FILLED]

Report status: Academic draft. No newly measured Gemini benchmark or signed declaration is asserted.

1. Problem statement and users

AUTOSAR-like High-Level Design (HLD) documents describe software components, provided/required ports, interfaces and architectural data flow. Information is spread across pages and tables. Manual reading is slow; simple search lacks paraphrase tolerance; unrestricted LLMs can fabricate precise engineering facts. Intended users are automotive software engineers, architecture reviewers, systems/integration engineers and academic evaluators. The goal is grounded document Q&A plus structured architecture exploration, with human oversight.

2. Input data and knowledge base

Text-based HLD PDF files enter via upload. The repository contains synthetic Braking System HLD (../data/samples/AUTOSAR_HLD_Sample_BrakingSystem.pdf) and ten-question golden dataset (../data/golden_dataset.json) (seven answerable, three unanswerable). A separate Door Control HLD was manually tested but is not checked into this repo. ChromaDB stores embedded chunks and evidence metadata; SQLite stores SWC/Port/Interface entities. Runtime databases are regenerated from inputs, not model-training data.

3. Architecture / workflow

Ingestion: Streamlit → FastAPI → coordinator → PyMuPDF page/block extraction → cleaning → hierarchical chunking → all-MiniLM-L6-v2 embeddings → ChromaDB, plus deterministic entity extraction → Pydantic validation/deduplication → SQLite.

Question answering: user query → embedding → semantic top-k retrieval → evidence-delimited grounded prompt → MockLLM or Gemini → citation ID verification → Streamlit answer, citations, page/section/snippet, grounding warnings.

Architecture browsing: Streamlit → FastAPI entities endpoint → SQLite → SWC/Port/Interface tables. See Mermaid architecture (ARCHITECTURE_AND_DATA_FLOW.md).

4. Implementation and configuration

The project is Python-based with separate core, backend, frontend, storage, config and tests packages. PyMuPDF preserves block and page metadata; TextCleaner handles repeated headers/footers, hyphenation and whitespace; HierarchicalChunker produces provenance-linked text chunks.

The pretrained sentence transformer all-MiniLM-L6-v2 converts chunk/query text to semantic vectors. ChromaDB returns nearest chunks, ordered by similarity. The normal coordinator uses

top-k 5; its relevance cutoff defaults to None. GroundedPromptBuilder limits context to 12,000 chars, wraps document text in untrusted <context> / <chunk> regions, requests [chunk_id] citations and requires explicit insufficient evidence for absent information.

BaseLLMProvider enables deterministic MockLLM and a Gemini provider. The coordinator supplies generation temperature 0.0. Actual Gemini model is environment-dependent: config/settings.py defaults to gemini-3-flash-preview, while the tracked .env.example selects gemini-2.5-flash. Actual run settings need independent confirmation. Never commit the actual key.

CitationVerifier rejects unknown cited IDs, resolves known IDs to document/page/section/snippet and labels explicit insufficient-evidence responses. It does not semantically prove the answer. This distinction is necessary because the earlier MockLLM yielded generic cited outputs even when a specific factual response or negative refusal was expected.

EntityExtractor identifies SWCs, Ports and Interfaces using patterns and table parsing, validates entities and deduplicates before SQLite persistence. Optional LLM fallback exists, but the upload coordinator explicitly sets use_llm=False, so default ingestion is rules-based. FastAPI exposes GET /health, POST /documents/upload, POST /rag/query and GET /entities/{doc_id}; Streamlit supplies the workbench.

5. Evaluation steps and results

The repository includes tests covering configuration, PDF parsing/cleaning, chunking, embeddings, Chroma, retrieval, prompt builder, mock/provider integration, citations, extraction, evaluation, FastAPI and UI. The golden dataset contains 10 questions; PipelineEvaluator (../core/evaluation/evaluator.py) calculates retrieval success, citation-ID validity, groundedness flags, negative-grounding and keyword-overlap ratios. These are implementation-defined proxies and must not be represented as human-verified factual accuracy.

A student earlier reported 70 passed, 1 skipped (historical) and 16 indexed chunks on a Door Control test (historical). Fresh automated tests were not executed as part of this documentation commit. The tested MockLLM produced a generic cited answer to an unsupported MCU query, so a passing negative-grounding test is not established. Protocol and status (../Evaluation_Results/EVALUATION_STATUS.md) document remaining evidence.

6. Responsible AI, privacy and security

Evidence-restricted prompting; citation-ID checks with human source review; explicit abstention requirement; untrusted-document delimiter handling; configuration via private .env; avoiding confidential input PDFs in public GitHub; student responsibility for technical conclusions. Safety limits: valid IDs do not entail factual claims, PDF prompt-injection defenses are partial, externally hosted providers may receive engineering content, and no compliance/ASIL claim has been established. This software is advisory.

7. Innovation highlights

1. Traceable source chunks with page/section/snippet.

2. Deterministic citation-ID validation and warnings.

3. AUTOSAR-specific SWC/Port/Interface extraction.

4. Dual databases: ChromaDB semantic index and SQLite structured entities.

5. Modular local MockLLM / Gemini providers with testability.

6. End-to-end Streamlit + FastAPI workbench rather than raw LLM chat.

8. Limitations and future scope

Text-only PDF assumptions; irregular tables/diagrams; exact identifier retrieval; no OCR/ARXML in core path; no authenticated tenant isolation; citation validity not semantic entailment. Future enhancements: hybrid BM25+vector retrieval, graph relationships, ARXML, measured benchmark coverage, entailment testing, secured enterprise deployment.

9. Conclusion

The project demonstrates domain-specific integration of pretrained ML embeddings and a generative LLM with evidence retrieval, provenance validation and structured automotive architecture extraction. It is a working prototype intended to assist, not replace, human engineering review.

10. Referenced sources

• Project code: https://github.com/SABA718/AUTOSAR-HLD-AI-Assistant

• AUTOSAR: https://www.autosar.org/

• PyMuPDF: https://pymupdf.readthedocs.io/

• Sentence Transformers: https://www.sbert.net/

• Chroma: https://docs.trychroma.com/

• Gemini: https://ai.google.dev/gemini-api/docs

• FastAPI: https://fastapi.tiangolo.com/

• Streamlit: https://docs.streamlit.io/

Architecture and Workflows

flowchart TD

UI[Streamlit Engineering Workbench] --> API[FastAPI REST]

API --> C[PipelineCoordinator]

C --> PDF[PyMuPDF PDF extraction]

PDF --> CLEAN[Text cleaning]

CLEAN --> CHUNK[Hierarchical chunking]

CHUNK --> EM[Sentence-Transformer embeddings]

EM --> DB[(ChromaDB vector index)]

CHUNK --> ENT[SWC/Port/Interface rule extraction]

ENT --> SQL[(SQLite structured entities)]

C --> RET[Semantic Retriever top-k]

RET --> DB

RET --> PROMPT[Grounded Prompt Builder]

PROMPT --> LLM[MockLLM or Gemini]

LLM --> VERIFY[Citation ID Verifier]

VERIFY --> API

SQL --> API

Ingestion: PDF → page/block text → cleaning → chunks → embeddings → vector index + deterministic entity extraction → SQLite.

Q&A: question → query embedding → ChromaDB nearest neighbors → grounded prompt → provider → cited answer → Python citation validation → answer/evidence in Streamlit.

Source of truth: backend/coordinator.py (../backend/coordinator.py), core/rag (../core/rag), frontend/app.py (../frontend/app.py).

The citation check validates the *reference ID*, not semantic entailment. Human reviewers must open the cited source before accepting engineering claims.

Implementation Module Reference

Module	Primary responsibility
config/settings.py	Pydantic BaseSettings, environment overrides and storage paths
core/logger.py	Structured logging
core/ingestion/pdf_extractor.py	PyMuPDF document/pages/blocks extraction
core/ingestion/cleaner.py	Hyphenation, whitespace, recurring header/footer cleaning
core/chunking/hierarchical_chunker.py	Structural/section chunks
core/embeddings/embedder.py	Pretrained Sentence Transformers inference
core/vector_store/chroma_manager.py	Chroma persistent index, vector lookups
core/retrieval/retriever.py	Ranked chunks, top-k and optional threshold
core/rag/prompt_builder.py	Evidence delimitation and grounding instructions
core/llm/base.py	Provider-neutral response interface
core/llm/mock_provider.py	Deterministic mock response
core/llm/gemini_provider.py	Gemini inference
core/llm/factory.py	Provider selection
core/rag/citation_engine.py	Valid/invalid citation ID mapping
core/extraction/entity_extractor.py	SWC/Port/Interface candidates, validation, deduplication
storage/database.py	SQLite access/upserts
backend/coordinator.py	Ingest, query, entities orchestration
backend/main.py	FastAPI endpoints
frontend/app.py	UI upload, question and entity tables
core/evaluation/evaluator.py	Dataset-based proxy metrics
tests	Component/API/UI regression tests

Input and Preprocessing

1. Ingest a digital-text HLD PDF with PyMuPDF; preserve 1-indexed page positions, text blocks and headings/layout when available.

2. Normalize line-wrap hyphenation, repetitive headers, margin page number footers and whitespace; retain necessary structural and table boundaries.

3. Split pages into hierarchical/section chunks; retain source document, page, section and deterministic chunk ID for later citations.

4. Encode chunks using pretrained all-MiniLM-L6-v2; persist vectors/chunk text/source metadata in ChromaDB.

5. Parse structured AUTOSAR names and common tables, validate/deduplicate SWCs/Ports/Interfaces and upsert into SQLite.

Included test artifacts: synthetic Braking System PDF (../data/samples/AUTOSAR_HLD_Sample_BrakingSystem.pdf) and ten golden questions (../data/golden_dataset.json). Door Control sample mentioned in manual tests is not currently in this GitHub repository.

Limitations: image-only scans need OCR, unusual multi-column layouts and diagrams can lose source relationships. No PCA/t-SNE/UMAP stage is used.

AI/ML Algorithms, Dimensionality and Metrics

ML method: pretrained transformer encoder all-MiniLM-L6-v2 produces semantic vectors for both retrieved chunks and questions. Retrieval: ChromaDB vector similarity ranks candidate evidence. Generative model: Gemini synthesizes a grounded-language answer; MockLLM substitutes deterministic text in tests. Domain logic: structured AUTOSAR extraction and citation ID verification are deterministic rule-based components.

Dimensionality reduction: no separately fitted PCA, t-SNE or UMAP. Learning an embedding is not equivalent to applying PCA.

Threshold: normal default top_k=5, relevance threshold None. Top-k controls result count; a cutoff filters weak matches. Similarity scores are not probabilities of truth. Evaluator has a 0.3 default argument under a distinct evaluation path; do not confuse this with application default.

Evaluation available in source: retrieval success, citation ID validity, implementation-defined groundedness rate, negative-grounding accuracy and expected-keyword overlap. They are limited: a correctly cited generic answer can be useless or unsupported. Manual comparisons with the actual PDF are essential.

Why AI matters: semantic retrieval beyond exact keywords and engineering-text generation based on local documentary evidence; source provenance and testing are software engineering contributions. No newly trained model claimed.

Responsible AI, Privacy, Security, Human Oversight

• Grounding: prompt confines generation to supplied retrieved text and mandates exact chunk IDs.

• Abstention: explicit insufficient-evidence phrase for information absent from evidence; validate real behavior before claiming reliable refusal.

• Traceability: source document, page, section, chunk ID and evidence snippet are shown to users.

• Verifier limitations: syntactic citation-ID validity does not prove every generated claim is supported or logically implied by the source.

• Prompt injection: source text is lower-trust and delimiters are sanitized, but this is not a complete defense.

• Data privacy: only use authorized HLDs; a remote Gemini call may send extracted proprietary content to an external provider. Do not commit private OEM data or .env.

• Key handling: store credentials in private environment files or secret vaults; avoid logs and screenshots with keys.

• Human review: an automotive engineer must verify architecture/safety claims; no ASIL compliance or production safety assurance is certified.

• Reporting integrity: distinguish test expectations, MockLLM outputs and verified real Gemini results. Disclose failures; never fabricate metrics.

Potential future controls include tenant isolation, access permissions, retention policies and claim-level evidence verification.

Installation and Demonstration — Windows PowerShell

From repository root:

python -m venv .venv

.\.venv\Scripts\Activate.ps1

python -m pip install -r requirements.txt

Copy-Item .env.example .env

Edit .env locally. Start with DEFAULT_LLM_PROVIDER=mock.

python -m pytest -q

Terminal 1 — API:

python -m uvicorn backend.main:app --reload

Open http://127.0.0.1:8000/health and http://127.0.0.1:8000/docs.

Terminal 2 — UI:

.\.venv\Scripts\Activate.ps1

python -m streamlit run frontend/app.py

Open http://localhost:8501. Upload data/samples/AUTOSAR_HLD_Sample_BrakingSystem.pdf; inspect metrics, SWC/Port/Interface tables; ask a document-answerable question, open verified citations; ask an absent-fact question and inspect whether it refuses.

Real Gemini: store GEMINI_API_KEY privately in .env; select DEFAULT_LLM_PROVIDER=gemini and an available model; restart both services. Capture the provider label, factual answer and source evidence without leaking secrets.

MockLLM is an integration test double, not a measure of natural answer quality. Real LLM results and negative grounding require human inspection.

Viva and Interview Questions

What is the project? A document-grounded AUTOSAR HLD assistant for engineering Q&A and structured SWC/Port/Interface exploration.

Why RAG? Index changing PDFs and retrieve source evidence without retraining model weights; expose source citations.

Which AI algorithms? Pretrained transformer embeddings (all-MiniLM-L6-v2), vector similarity retrieval, and a Gemini generative model.

Did you train a model or apply PCA? No. The project uses pretrained models; no PCA/t-SNE/UMAP stage is implemented.

What are preprocessing steps? PyMuPDF block/page extraction, header/footer and hyphen cleaning, whitespace normalization, hierarchical chunking.

What is top-k? Number of candidate nearest chunks (5 by default).

What is the relevance threshold? Minimum retrieval similarity if enabled. Application default is None, so not always applied.

Why two databases? ChromaDB for semantic chunks/embeddings; SQLite for structured SWCs/Ports/Interfaces.

What exactly does Gemini do? Converts relevant retrieved HLD text into a readable answer; it is not the PDF retriever.

What does citation verification prove? Only that cited IDs point to retrieved source; a separate human check is needed for actual claim support.

What is an extra feature over simple PDF chatbot? Domain-aware entity extraction and architecture explorer, source provenance, deterministic citations and offline MockLLM testing.

How do you evaluate? Tests, golden questions, retrieval/citation-ID metrics, human factual support, refusal correctness and latency.

What failed during testing? A generic MockLLM answer with a valid citation was observed even for a missing-MCU question; it is a limitation, not a successful negative-grounding demonstration.

What next? Hybrid lexical+vector search for AUTOSAR identifiers, ARXML support, graph relations and stronger factual-entailment checks.

Final Academic Submission Audit

- [ ] Eight named folders exist at repository root.
- [ ] Student, register number, institution filled in by student.
- [ ] Printable submissions use Times New Roman 12pt.
- [ ] Run and preserve a dated python -m pytest -q result.
- [ ] Record model ID, provider, top-k and threshold actually used.
- [ ] Benchmark Braking System PDF against data/golden_dataset.json.
- [ ] Test real Gemini positive answer and unknown-fact refusal.
- [ ] Inspect citations against PDF pages/sections, not only ID validity.
- [ ] Include real logs/screenshots and failures, without keys.
- [ ] Keep API keys, .env, ChromaDB/SQLite runtime data and private documents out of repo.
- [ ] Update any discrepancy between code and documentation.
- [ ] Review model/tool attribution and source citations.
- [ ] Sign academic integrity and AI-tool usage declarations.
- [ ] Add video recording when available.
- [ ] Make and verify final ZIP according to faculty instructions.

External Source / Model Register

Verify library licenses and actual usage before submitting:

Source	Purpose	Reference
AUTOSAR	Automotive architecture standards/background	https://www.autosar.org/
PyMuPDF	PDF parsing	https://pymupdf.readthedocs.io/
Sentence Transformers	Pretrained MiniLM model	https://www.sbert.net/
Chroma	Vector database	https://docs.trychroma.com/
Google Gemini API	External text generation model	https://ai.google.dev/gemini-api/docs
FastAPI	API services	https://fastapi.tiangolo.com/
Streamlit	Frontend	https://docs.streamlit.io/
Synthetic Braking System HLD	Educational source document	Repository sample (../data/samples/AUTOSAR_HLD_Sample_BrakingSystem.pdf)

No official OEM/production HLD content should be published without authorization.